



UNIDADE III

Programação Orientada
a Objetos com C#

Prof. MSc. Tarcísio Peres

Conteúdo da Unidade III

Coleções e genéricos

- List<T>, Dictionary<TKey,TValue>, iteração, IEnumerable<T>
- Genéricos: restrições, igualdade/ordenação e boas práticas

LINQ essencial

- Where/Select/GroupBy/Any, projeções e diferimento de execução
- IEnumerable<T> × IQueryable (noções para acesso a dados)

List<T>, Dictionary<TKey,TValue> e iteração

- Coleções genéricas mais usadas em C#
- Segurança de tipo em tempo de compilação
- Iteração uniforme com foreach via IEnumerable<T>

```
internal class Program
{
    static void Main(string[] args)
    {
        // Cria uma lista de produtos (List<Produto>)
        List<Produto> catalogo = new List<Produto>
        {
            new Produto(1, "Teclado mecânico", 350.00m),
            new Produto(2, "Mouse gamer", 150.00m),
            new Produto(3, "Monitor 24\"", 900.00m),
            new Produto(4, "Headset", 250.00m)
        };

        Console.WriteLine("=== catálogo inicial (List<Produto>) ===");
        ExibirCatalogo(catalogo);

        Console.WriteLine();
        Console.WriteLine($"Count da lista: {catalogo.Count}");
        Console.WriteLine($"Capacity da lista: {catalogo.Capacity}");
    }
}
```

System.Collections.Generic

- List<T>: sequência ordenada (array dinâmico)
- Dictionary<TKey,TValue>: mapa chave→valor (tabela hash)
- Parâmetros de tipo (T, TKey, TValue) fixam o tipo dos elementos
- Integram com LINQ por implementarem IEnumerable<T>

Segurança de tipo: menos cast e menos erros

- `List<string>` aceita apenas string (checado pelo compilador).
- Coleções não genéricas (ex.: `ArrayList`) armazenam object.
- Ao recuperar, é comum precisar de cast → risco de `InvalidCastException`.
- cast é uma conversão explícita de tipo que você escreve no código usando parênteses, para dizer ao compilador: “trate este valor como sendo do tipo X”: `(Tipo)expressão`.
- Genéricas evitam casts explícitos na maioria dos casos.

Boxing e unboxing

- Tipos por valor (int, double, bool, DateTime) não são objetos.
- Ao armazenar um value type em uma variável do tipo object ou em uma coleção cujos elementos são do tipo object, ocorre boxing (alocação no heap).
- Ao ler de volta, ocorre unboxing (verificação + cópia) e pode falhar em runtime.
- List<int> armazena int diretamente → sem boxing/unboxing.

List<T> cresce automaticamente

- Arrays têm tamanho fixo; é difícil prever quantos itens existirão.
- List<T> mantém um array interno redimensionável.
- Add() aumenta Count; se Count ultrapassar Capacity há realocação e cópia.
- Count = itens atuais | Capacity = espaço alocado.
- Para reduzir memória, pode-se usar TrimExcess() quando apropriado.

```
List<Produto> catalogo = new List<Produto>
{
    new Produto(1, "Teclado mecânico", 350.00m),
    new Produto(2, "Mouse gamer", 150.00m),
    new Produto(3, "Monitor 24\"", 900.00m),
    new Produto(4, "Headset", 250.00m)
};

Console.WriteLine("=== catálogo inicial (List<Produto>) ===");
ExibirCatalogo(catalogo);

Console.WriteLine();
Console.WriteLine($"Count da lista: {catalogo.Count}");
Console.WriteLine($"Capacity da lista: {catalogo.Capacity}");

// Adiciona mais um produto: a lista cresce conforme necessário
catalogo.Add(new Produto(5, "Webcam Full HD", 300.00m));
```

foreach, IEnumerable<T> e IEnumerator<T>

- foreach funciona com qualquer objeto que implemente IEnumerable<T>.
- Nos bastidores, chama GetEnumerator() para obter um IEnumerator<T>.
- O enumerador avança com MoveNext() e expõe o item atual em Current.
- A iteração independe do armazenamento: List, Dictionary ou iterators.

```
List<int> numeros = new List<int>();  
numeros.Add(5);  
numeros.Add(10);  
numeros.Add(15);
```

```
foreach (int n in numeros) {  
    Console.WriteLine(n);  
}
```


Dictionary<TKey,TValue>: acesso por chave

- Armazena pares chave-valor (KeyValuePair<TKey,TValue>).
- A chave (TKey) é única: não existem duas entradas com a mesma chave.
- Implementação típica usa tabela hash baseada em GetHashCode().
- Ideal para localizar rapidamente um valor quando a chave é conhecida.

```
// Cria um Dictionary<int, Produto> para acesso rápido por Id
Dictionary<int, Produto> produtosPorId = new Dictionary<int,
Produto>();

foreach (var produto in catalogo)
{
    produtosPorId[produto.Id] = produto;
}

Console.WriteLine();
Console.Write("Digite um id de produto para buscar: ");
```

Operações e iteração em Dictionary

- Inserir/atualizar: dict[chave] = valor
- Ler: dict[chave] (lança exceção se a chave não existir)
- Busca segura: TryGetValue(chave, out valor)
- Iterar: foreach (var par in dict) → par.Key / par.Value
- Também é possível iterar dict.Keys e dict.Values

```
Dictionary<string,int> idades = new Dictionary<string,int>();  
idades["Alice"] = 30;  
idades["Bob"] = 25;
```

```
foreach (KeyValuePair<string,int> par in idades) {  
    Console.WriteLine($"{par.Key}: {par.Value}");  
}
```

IEnumerable<T> e iterators com yield return

- Métodos podem retornar IEnumerable<T> para expor sequências.
- Implementar IEnumerable<T> torna uma classe compatível com foreach.
- yield return cria um iterator sem escrever IEnumerator<T> manualmente.
- Execução adiada: os valores são produzidos conforme são iterados.

```
static IEnumerable<Produto> FiltrarPorPrecoMinimo(  
    IEnumerable<Produto> produtos,  
    decimal precoMinimo)  
{  
    foreach (var produto in produtos)  
    {  
        if (produto.Preco >= precoMinimo)  
        {  
            yield return produto;  
        }  
    }  
}
```

Interatividade

Qual alternativa descreve corretamente uma vantagem das coleções genéricas?

- a) `ArrayList` evita boxing de tipos por valor.
- b) `List<int>` armazena `int` sem boxing/unboxing.
- c) `Dictionary<TKey,TValue>` permite chaves duplicadas.
- d) `foreach` só funciona com arrays.
- e) Nenhuma das alternativas anteriores está correta.

Resposta

Qual alternativa descreve corretamente uma vantagem das coleções genéricas?

- a) ArrayList evita boxing de tipos por valor.
- b) `List<int>` armazena int sem boxing/unboxing.
- c) Dictionary<TKey,TValue> permite chaves duplicadas.
- d) foreach só funciona com arrays.
- e) Nenhuma das alternativas anteriores está correta.

Genéricos no C#

- Os parâmetros de tipo (T) permitem reutilizar classes, métodos e interfaces com diferentes tipos.
- O compilador verifica a consistência de tipos em tempo de compilação.
- Evita duplicação de código e mantém boa performance.

Por que usar restrições (where)

- As restrições limitam o conjunto de tipos possíveis para T.
- Permitem supor contratos e membros sem testes em tempo de execução.
- Erros aparecem no compilador, reduzindo validações manuais.

Restrições básicas: class e struct

- where T : class → apenas tipos de referência (classes, interfaces, arrays e string).
- where T : struct → tipos valor não anuláveis (int, double e DateTime).
- A escolha afeta nulidade, semântica e interoperabilidade com genéricos.

```
class Caixa<T> where T : class { } // só aceita tipos de referência (classes,  
interfaces, delegates etc.)
```

```
class Caixa2<T> where T : struct { } // só aceita tipos por valor
```


Restrição new()

- where T : new() exige um construtor público sem parâmetros.
- Permite instanciar T dentro do código genérico: new T().
- É útil em fábricas, repositórios e inicialização-padrão.

Restrições por herança e por interface

- where T : MinhaClasseBase garante, no mínimo, os membros definidos na classe base.
- where T : IMinhaInterface permite chamadas diretas aos métodos da interface, sem casts.
- O contrato do tipo fica explícito e verificável em compilação.

Combinação e ordem das restrições

- É possível combinar restrições: `where T : MinhaClasseBase, IComparable<T>, new()`.
- Isso exige herança, implementação de interface e construtor sem parâmetros.
- Regra prática: a classe base vem antes das interfaces; `new()` fica por último.

Exemplo: Repositorio<T>

- Repositorio<T> usa where T : EntidadeBase, new().
- EntidadeBase fornece a propriedade Id; new() viabiliza o método CriarENovo().
- Repositorio<Cliente> compila porque Cliente deriva de EntidadeBase e tem construtor público sem parâmetros.

Igualdade tipada com `IEquatable<T>`

- Cliente implementa `IEquatable<Cliente>` via `Equals(Cliente other)`.
- A igualdade considera `Id` e `Nome`; `ReferenceEquals` trata nulos e autocomparação.
- `Equals(object)` e `GetHashCode` são sobrescritos para manter consistência em coleções.

Ordenação: `Comparable<T>` e utilitário genérico

- Cliente implementa `Comparable<Cliente>`; `CompareTo` define a ordenação natural (por Nome).
- `List<Cliente>.Sort()` usa `Comparable<T>` para ordenar os elementos.
- `EncontrarMaximo<T>` exige where `T : Comparable<T>` para permitir `item.CompareTo(maximo)`.

Comparadores externos e boas práticas

- `IComparer<Cliente>` permite ordenação alternativa (`ClientePorIdComparer`), sem alterar a ordenação natural por Nome.
- `IEqualityComparer<Cliente>` define igualdade externa (por Nome, ignorando maiúsculas/minúsculas) para `HashSet` e `Dictionary`.
- Boas práticas: nomeação clara; preferir `where a is/as`; evitar generalização excessiva no desenho do tipo genérico.

Interatividade

Em um código genérico que cria instâncias com `new T()`, qual restrição é obrigatória?

- a) `where T : class`
- b) `where T : struct`
- c) `where T : new()`
- d) `where T : IComparable<T>`
- e) Nenhuma das anteriores está correta.

Resposta

Em um código genérico que cria instâncias com `new T()`, qual restrição é obrigatória?

- a) `where T : class`
- b) `where T : struct`
- c) `where T : new()`
- d) `where T : IComparable<T>`
- e) Nenhuma das anteriores está correta.

LINQ: o que é

- O LINQ integra consultas ao C# (consultas escritas como código tipado).
- Ele consulta coleções, bancos de dados, XML e outras fontes sem depender de strings soltas.
- Ele usa operadores como Where, Select, OrderBy e GroupBy sobre sequências.

As peças centrais do LINQ

- As query expressions usam from, where, orderby e select.
- Os operadores-padrão são métodos de extensão em System.Linq (Enumerable/Queryable).
- O compilador traduz a sintaxe de consulta em chamadas encadeadas de métodos.

```
var q = from n in numeros
where n % 2 == 0
orderby n descending
select n * n;
```

LINQ to Objects

- Ele opera sobre coleções em memória que implementam `IEnumerable<T>` (`List<T>`, arrays).
- A consulta costuma executar ao enumerar (foreach), por execução adiada.
- A consulta compõe um pipeline de filtragem, ordenação e projeção.

```
var q = numeros.Where(n => n % 2 == 0)
               .OrderByDescending(n => n)
               .Select(n => n * n);
```

Lambdas em LINQ

- Where recebe um predicado (Func<TSource, bool>).
- Select recebe uma função de projeção (Func<TSource, TResult>).
- A expressão lambda (=>) fornece essas funções de forma concisa.

```
var pares = numeros.Where(n => n % 2 == 0);  
var quad = pares.Select(n => n * n);
```

Métodos de extensão e o operador .

- `numeros.Where(...)` equivale a `Enumerable.Where(numeros, ...)`.
- Os operadores aparecem como membros por serem métodos de extensão.
- É comum importar `System.Linq` para habilitar esses métodos.

```
using System.Linq;  
var r = Enumerable.Where(numeros, n => n > 0);
```

Execução adiada e imediata

- Where e Select, em geral, são adiados: executam ao enumerar.
- Se a fonte muda antes do foreach, o resultado pode mudar.
- ToList(), ToArray() e Count() forçam execução imediata.

```
var q = lista.Select(n => n * 10);  
var l = q.ToList(); // executa agora
```

Where

- Where filtra e emite somente os elementos que satisfazem a condição.
- Ele retorna `IEnumerable<T>`, sem modificar a sequência de origem.
- A condição costuma ser uma lambda, por exemplo, `n => n % 2 == 0`.

```
var pares = numeros.Where(n => n % 2 == 0);
```


Select

- Select projeta: transforma cada elemento em outro valor.
- Ele pode mudar o tipo (por exemplo, `int` → `string`) ou criar tipos anônimos.
- Como Where, ele retorna `IEnumerable<T>` e não altera a fonte.

```
var nomesMaius = nomes.Select(n => n.ToUpper());
```

GroupBy e IGrouping

- GroupBy agrupa por uma chave e retorna IGrouping<TKey, TElement>.
- Cada grupo expõe Key e é enumerável (os elementos do grupo).
- O agrupamento é montado ao enumerar o resultado (adiado).

```
var g = nums.GroupBy(n => n % 2 == 0 ? "Par" : "Ímpar");
```

Any, IEnumerable e IQueryable

- Any testa existência: Any(predicado) ou Any() para sequência não vazia.
- Em IEnumerable<T> a consulta executa no CLR (delegates).
- Em IQueryable<T> a consulta pode virar árvore de expressão para um provedor (ex.: EF).

Interatividade

Qual operador força execução imediata em LINQ to Objects?

- a) Where(...)
- b) Select(...)
- c) ToList()
- d) OrderBy(...)
- e) GroupBy(...)

Resposta

Qual operador força execução imediata em LINQ to Objects?

- a) Where(...)
- b) Select(...)
- c) **ToList()**
- d) OrderBy(...)
- e) GroupBy(...)

LINQ e uma sintaxe comum de consulta

- LINQ permite consultar sequências que implementam `IEnumerable<T>` ou `IQueryable<T>`.
- Mesma sintaxe (Where, Select etc.) para coleções locais e fontes remotas.
- Diferença central: como e onde a consulta é executada (memória vs. servidor).

IEnumerable<T>: sequência iterável (LINQ to Objects)

- Representa uma sequência de elementos que pode ser enumerada (foreach).
- Em LINQ to Objects, os operadores-padrão ficam na classe System.Linq.Enumerable.
- Lambdas são compiladas como delegados e executadas localmente no código do programa.

IEnumerable<T>: execução adiada e filtro em memória

- Muitos operadores retornam uma nova sequência e só executam ao enumerar (deferred execution).
- O predicado do Where percorre os elementos em memória para aplicar o filtro.

```
List<Produto> lista = ObterListaProdutos();  
var caros = lista.Where(p => p.Preco > 100.0m); // Filtra em memória quando o  
resultado é enumerado
```


IQueryable<T>: consulta como árvore de expressão

- Uma árvore de expressão (expression tree) é uma estrutura de dados que representa um trecho de código (uma expressão) como objetos em vez de como código executável direto.
- Estende IEnumerable<T> para representar consultas como expression trees.
- Os operadores de consulta são resolvidos para System.Linq.Queryable.
- Um provedor interpreta a expressão (ex.: Entity Framework) para executar na fonte.
- Exemplo: `p => p.Preco >= precoMinimo`
- Lambda
 - — Parâmetro: `p`
 - — Corpo: `(>=)`
 - — Membro: `p.Preco`
 - — Variável/constante: `precoMinimo`

Exemplo: tradução para SQL (LINQ to Entities)

- A árvore de expressão pode ser traduzida para uma consulta nativa (como SQL).
- O filtro roda no servidor e retorna somente os registros necessários.

```
var clientesDoBrasil = contexto.Clientes  
    .Where(c => c.Pais == "BR" && c.Ativo == true); // Provedor traduz para  
    SQL e executa no banco
```

Quando a consulta é executada?

- Em `IQueryable<T>`, compor a consulta não executa o acesso à fonte de dados.
- A execução ocorre ao enumerar ou materializar (`ToList`, `First`, `Count` etc.).
- Isso permite adicionar filtros/projeções antes de enviar a consulta ao servidor.

Armadilha: cair para o cliente (Enumerable)

- `IQueryable<T>` herda `IEnumerable<T>`, mas trocar para operadores de `Enumerable` pode mudar o modo de execução.
- `AsEnumerable()/ToList()` materializam: dali em diante, o restante roda em memória.
- Consequência típica: mais dados trafegados e maior consumo de memória no cliente.

Desempenho e cenários de uso

- Prefira `IQueryable<T>` para fontes externas/volumosas: delega filtro e agregações ao servidor.
- Use `IEnumerable<T>` para coleções já carregadas em memória (ou fontes sem provedor traduzível).
- Regra prática: quanto maior o dado remoto, mais importante filtrar “antes” de materializar.

Convertendo com segurança (ToList / AsEnumerable)

- Às vezes, é necessário: trazer um subconjunto e aplicar regras de negócio com LINQ to Objects.
- Use ToList()/AsEnumerable() conscientemente e o mais tarde possível na composição.
- Evite cadeias longas em memória após materializar: risco de CPU e RAM.

Comparação

IEnumerable<T> (local)

- Dados já carregados
- Filtro em memória
- Sem tradução para SQL

```
List<Pedido> pedidos = Repo.ObterPedidos();  
var recentes = pedidos  
    .Where(p => p.Data >  
        DateTime.Today.AddMonths(-1));
```

IQueryable<T> (remoto)

- Consulta composta
- Traduzível pelo provedor
- Executa no servidor

```
var pedidosEF = contexto.Pedidos; // IQueryable  
var recentes = pedidosEF  
    .Where(p => p.Data >  
        DateTime.Today.AddMonths(-1));  
// SQL só ao materializar
```

Interatividade

Qual alternativa descreve melhor a diferença entre `IEnumerable<T>` e `IQueryable<T>`?

- a) `IEnumerable` traduz para SQL; `IQueryable` executa sempre em memória.
- b) `IEnumerable` é sequência iterável; `IQueryable` representa a consulta como expressão para um provedor.
- c) Ambos sempre executam no servidor.
- d) `IQueryable` não suporta execução adiada.
- e) `IEnumerable` só funciona com bancos de dados.

Resposta

Qual alternativa descreve melhor a diferença entre `IEnumerable<T>` e `IQueryable<T>`?

- a) `IEnumerable` traduz para SQL; `IQueryable` executa sempre em memória.
- b) `IEnumerable` é sequência iterável; `IQueryable` representa a consulta como expressão para um provedor.
- c) Ambos sempre executam no servidor.
- d) `IQueryable` não suporta execução adiada.
- e) `IEnumerable` só funciona com bancos de dados.

ATÉ A PRÓXIMA!